

CIM シミュレータ コード詳解(`modules/CIM.py`)

本資料は CIM(Coherent Ising Machine)シミュレータの実装 `modules/CIM.py` を、更新式の画像・論文の数式・実際のコード行の三者を突き合わせて詳細に解説する。「どの式の、どの項を、コードのどこで計算しているか」を一行ずつ対応させることを目的とする。

参照論文:Inoue & Yoshida, "Traveling-wave model of coherent Ising machine based on fiber loop with pulse-pumped phase-sensitive amplifier", Optics Communications **522** (2022) 128642.

1. CIM は物理的に何をしているか

CIM は 光ファイバーのループの中を N 個の光パルスが周回する装置である(`modules/CIM.py:10-20`)。ループの途中に PSA(位相感応増幅器) があり、パルスが 1 周するたびに:

1. PSA がポンプ光のエネルギーで in-phase 成分 c_i を増幅する(quadrature 成分は減衰)。縮退パラメトリック発振なので、発振パルスの位相は 0 か π の 2 値=振幅 c_i の符号 $\pm$ しか取れない。
2. MFB(測定フィードバック) が結合行列 J_{ij} を通じて他パルスからの入力 $\sum_j J_{ij}c_j$ を足し込む。
3. ASE+真空ノイズが加わる。

最終的に各 c_i の符号($\pm$)が Ising スピンになり、 $J_{ij} < 0$ (反強磁性)とすることで MAX-CUT(異なる集合に分けるほど得)を埋め込む。ポンプを毎ラウンド少しずつ上げる(ランプ=焼きなまし)ことで、ノイズで揺らぐ状態から良い解へ徐々に凍結させる。

2. 更新式(画像)の全体像

実装が 1 ラウンドで計算しているのは、次の 2 本の式である(画像 `docs/cim_equations.png`)。

$$\mathbf{F}(n) = a\mathbf{E}(n-1) + b\mathbf{J}\mathbf{E}(n-1) + c\mathbf{n}_0$$

$$\mathbf{E}(n) = \mathbf{F}(n) \exp\left[\alpha p(n) \left(1 - \beta |\mathbf{F}(n)|^2\right)\right]$$

$$\mathbf{F}(n) = a \mathbf{E}(n-1) + b \mathbf{J} \mathbf{E}(n-1) + c \mathbf{n}_0 \quad (\text{結合})$$

$$\mathbf{E}(n) = \mathbf{F}(n) \exp[\alpha p(n) (1 - \beta |\mathbf{F}(n)|^2)] \quad (\text{増幅・飽和})$$

意味を一言で言うと:

- 1 本目(F の式)=「集める」段。前ラウンドの振幅 $\mathbf{E}(n-1)$ を、損失 a をかけて残し($a\mathbf{E}$)、結合 $\mathbf{J}$ を通して他パルスから足し込み($b\mathbf{J}\mathbf{E}$)、ノイズ $\mathbf{cn}_0$ を加えた PSA への入力場 $\mathbf{F}(n)$ を作る。
- 2 本目(E の式)=「増やして頭打ちにする」段。 $\mathbf{F}(n)$ にポンプ利得 $\exp[\alpha p(n)(1 - \beta|\mathbf{F}|^2)]$ をかける。指数の中の符号で振る舞いが切り替わる:
- $|\mathbf{F}|^2$ が小さい(初期):中身 $\approx +\alpha p \rightarrow \exp[+]$ で増幅
- $|\mathbf{F}|^2 > 1/\beta$:中身が負へ反転 $\rightarrow \exp[-]$ で減衰=飽和

この「増幅 $\rightarrow$ 飽和」で振幅が頭打ちになり、位相が $0/\pi$ (符号 $\pm$)にロックしてスピンの確定する。

画像の各記号 $\leftrightarrow$ 実装の対応

画像の記号	意味	コードの変数	値・式
$\mathbf{E}(n)$	round n の振幅(=スピン候補)	<code>c[i]</code>	—
$\mathbf{E}(n-1)$	前ラウンドの振幅	<code>c[i]</code> (更新前)	—
$\mathbf{F}(n)$	結合後の場(PSA への入力)	<code>coupled_in</code>	$\sqrt{\eta} \cdot c + Jc$
a	損失後に残る自己項	<code>sqrt_eta</code>	$\sqrt{\eta}$
$b\mathbf{J}$	相互作用結合	<code>Jc</code>	J_{ij} (反強磁性)
$\mathbf{cn}_0$	ASE+真空ノイズ	<code>N_I</code> / <code>noise_i</code>	利得倍のガウスノイズ
$\alpha p(n)$	ポンプ項(利得)	<code>half_g0</code>	$\frac{1}{2}g_0 = \kappa L\sqrt{P}$
β	飽和係数	<code>gamma</code>	$\gamma = 42.09$
$ \mathbf{F}(n) ^2$	入力強度 I_{in}	<code>I_in</code>	<code>coupled_in²</code>
$\exp[\alpha p(n)(1 - \beta \mathbf{F} ^2)]$	in-phase 利得 $\sqrt{G_I}$	<code>sqrt_G_I</code>	<code>exp(half_g)</code>

ノイズの位置について:画像では $\mathbf{cn}_0$ が $\mathbf{F}(n)$ の中に入っているが、論文 Eq.(3a) と本実装ではノイズを増幅の後に加える(`c = sqrt_G_I * coupled_in + N_I`)。ただしノイズ強度自体が利得 $\sqrt{G_I}$ に比例する(`N_I = noise_const * sqrt_G_I * randn`)ため、「ノイズが利得倍される」という効果は両形式で一致する。

3. メインループのコード詳解(`simulate_cim`)

実際の 1 ラウンドの計算は [modules/CIM.py:316-341](#) にある(可読性の高い slow path を引用。JIT 版 `_simulate_cim_batch:96-119` も計算内容は同一)。以下、Step ごとに画像/論文式と対応づける。

Step 1 — ポンプ電力 → 非飽和利得係数 g_0

```
# modules/CIM.py:317-319
P_p = (k + 1) * dP_per_round
g0 = 2.0 * kappa * np.sqrt(P_p) * L
```

- **何をしているか:** ラウンド k のポンプ電力を決め、そこから PSA の非飽和利得係数 g_0 を計算する。
- **論文式 Eq.(14):** $g_0(k) = 2\kappa\sqrt{P_p(k)}L$ 。
- **画像との対応:** この $\frac{1}{2}g_0 = \kappa L\sqrt{P_p}$ が画像の **ポンプ項 $\alpha p(n)$** に相当する。
- **ポンプの増え方:** $P_p = (k + 1) \cdot dP$ なので電力はラウンドに線形。一方ポンプ項 $\alpha p(n) = \kappa L\sqrt{P_p} \propto \sqrt{k+1}$ は平方根(縮退パラメトリック増幅では利得が電場振幅 $\propto \sqrt{\text{電力}}$ に比例するため)。詳細は [CIM_pump_fixed_and_linear_vs_sqrt.md](#)。

Step 2 — 結合入力 $\mathbf{JE}(n-1)$ の計算(スパース行列ベクトル積)

```
# modules/CIM.py:323-324
Jc.fill(0.0)
csr_matvec(n, n, J_indptr, J_indices, J_data, c, Jc)
```

- **何をしているか:** 結合行列 $\mathbf{J}$ と現在の振幅ベクトル $\mathbf{c}$ の積 $\mathbf{Jc}$ を計算する(各パルスが他パルスから受け取る入力 $\sum_j J_{ij}c_j$)。
- **画像との対応:** F の式の $b\mathbf{JE}(n-1)$ 項。 $\mathbf{J}$ は `build_coupling_matrix` (modules/CIM.py:168-195) で構築され、G22 の辺に $J_{ij} = -0.03$ (反強磁性)が入っている。
- **実装の工夫:** scipy の `J @ c` はラッパーのオーバーヘッドが大きいので、低レベル `csr_matvec` を直接呼ぶ。`csr_matvec` は加算器なので、呼ぶ前に `Jc.fill(0.0)` で必ずゼロクリアする (modules/CIM.py:322)。JIT 版では CSR を手書きループで展開している (modules/CIM.py:104-110)。

Step 3 — PSA への入力場 $\mathbf{F}(n)$ と入力強度 $|\mathbf{F}|^2$

```
# modules/CIM.py:326-327
coupled_in = sqrt_eta * c + Jc
I_in = coupled_in * coupled_in
```

- **何をしているか:** 損失後の自己項 $\sqrt{\eta}\mathbf{c}$ に結合入力 $\mathbf{Jc}$ を足して、PSA への入力場 `coupled_in` を作る。その 2 乗が入力強度 `I_in`。
- **画像との対応:** `coupled_in` がそのまま $\mathbf{F}(n) = a\mathbf{E}(n-1) + b\mathbf{JE}(n-1)$ ($a = \sqrt{\eta}$ 、ノイズは後段で付与)。`I_in` が $|\mathbf{F}(n)|^2$ 。
- **論文式 Eq.(15):** $I_{\text{in}} \approx (\sqrt{\eta}c_i + \sum_j J_{ij}c_j)^2$ (真空ノイズと s_i は強度が小さいので無視)。
- **$\sqrt{\eta}$ の意味:** ループ損失 11 dB $\rightarrow \eta = 10^{-1.1} \approx 0.0794$ 。1 周ごとに信号パワーが η 倍に減衰する (modules/CIM.py:599-602)。

Step 4 — 飽和込みの利得 $\sqrt{G_I}$

```
# modules/CIM.py:332-333
half_g = 0.5 * g0 * (1.0 - gamma * I_in)
sqrt_G_I = np.exp(half_g)
```

- **何をしているか:**飽和を含めた利得係数 $g = g_0(1 - \gamma I_{\text{in}})$ を計算し、その指数 $\sqrt{G_I} = \exp(g/2)$ を求める。
- **画像との対応:**これが E の式の 指数部 $\exp[\alpha p(n)(1 - \beta |\mathbf{F}|^2)]$ そのもの。 $\alpha p(n) = \frac{1}{2}g_0$ 、 $\beta = \gamma$ 、 $|\mathbf{F}|^2 = I_{\text{in}}$ 。
- **論文式 Eq.(14):** $g = g_0(1 - \gamma I_{\text{in}})$, $G_I = \exp(g)$, $G_Q = \exp(-g)$ 。実装では G_I 自体ではなく $\sqrt{G_I} = \exp(g/2)$ を直接持つ(振幅更新もノイズ強度も $\sqrt{G_I}$ から出せるため、 `np.exp` 1 回で済ませる最適化)。
- **飽和の働き:** I_{in} が小さいうちは `half_g ≈ +½g0` で増幅、 $I_{\text{in}} > 1/\gamma$ になると `half_g` が負に転じて減衰 → 振幅が頭打ち(発振の飽和)。

Step 5 — ノイズ生成(ASE+真空ノイズ)

```
# modules/CIM.py:337
N_I = rng.standard_normal(n) * (noise_const * sqrt_G_I)
```

- **何をしているか:**標準正規乱数に標準偏差 $\sigma_I = \text{noise_const} \cdot \sqrt{G_I}$ をかけて、in-phase ノイズ N_I を生成する。
- **画像との対応:**F の式の $c \mathbf{n}_0$ 項(ただし実装では増幅後に加算)。
- **論文式 Eq.(6):** $\sigma_I^2 = (2 - \eta) G_I / 4 \cdot \text{BW}$ 。実装では $\sigma_I = \text{noise_const} \cdot \sqrt{G_I}$ と分解し、定数部 `noise_const = √((2-η)・0.25・BW・ħω)` を事前計算(`modules/CIM.py:298`)して毎ラウンドの `sqrt` を省く。
- **ポイント:**ノイズが $\sqrt{G_I}$ に比例する=「利得が大きいほどノイズも大きい」。初期($c = 0$)はこのノイズだけが種となり、自発的に振幅が立ち上がる(`modules/CIM.py:289-291`)。

Step 6 — 振幅更新 = $\mathbf{E}(n)$ の確定

```
# modules/CIM.py:341
c = sqrt_G_I * coupled_in + N_I
```

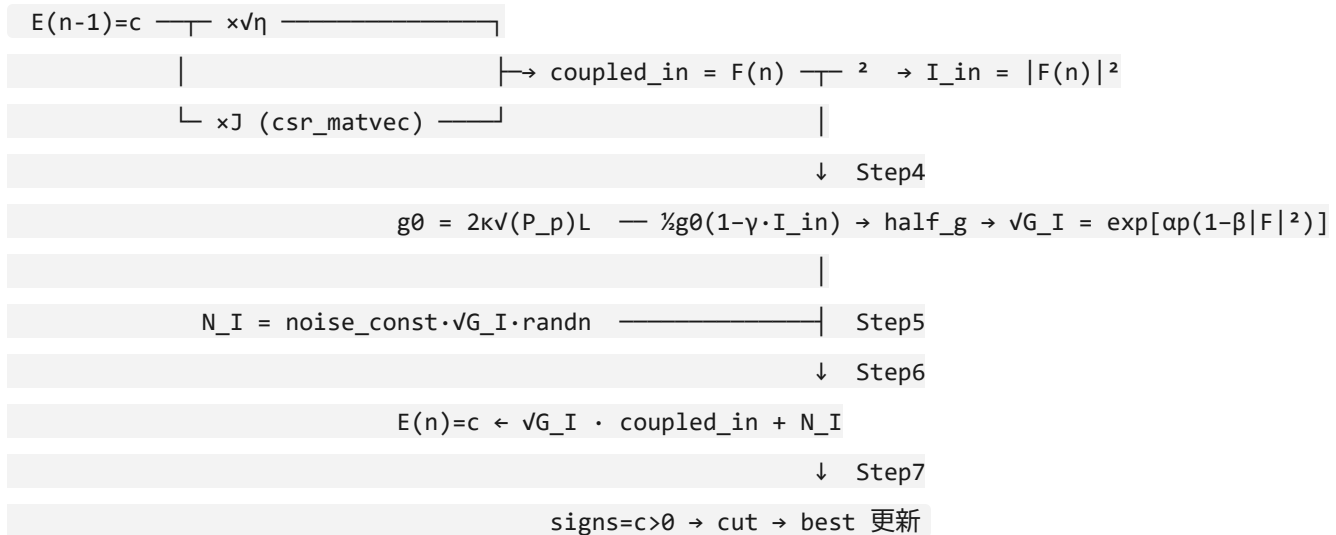
- **何をしているか:**利得 $\sqrt{G_I}$ をかけた入力場にノイズを足して、次ラウンドの振幅 c を確定する。
- **画像との対応:**これが E の式 $\mathbf{E}(n) = \mathbf{F}(n) \exp[\dots]$ の最終形(+ノイズ)。 `sqrt_G_I * coupled_in` が $\mathbf{F}(n) \cdot \exp[\dots]$ 、 `+ N_I` が画像の $c \mathbf{n}_0$ に相当。
- **論文式 Eq.(3a):** $c_i(k+1) = \sqrt{G_I}(\sqrt{\eta} c_i + \sum_j J_{ij} c_j) + N_I$ 。
- **quadrature 成分 s_i について:**論文 Eq.(3b) には s_i の式もあるが、 s_i は `coupled_in` にも cut 計算にも一切関与しないため、本実装では省略している(`modules/CIM.py:291-292`)。

Step 7 — cut 評価とベスト更新

```
# modules/CIM.py:344-348
signs = c > 0
cut = int((signs[edge_a] != signs[edge_b]).sum())
if cut > best_cut:
    best_cut = cut
    best_signs = signs.copy()
```

- **何をしているか:**各振幅の符号 $c_i > 0$ をスピン(0/1)とみなし、両端の符号が異なる辺の数=カット値を数える。過去最良を更新したら符号配列を保存する。
- **対応:**符号 $\rightarrow$ 0/1 は `amplitudes_to_solution ( modules/CIM.py:198-207 )` と同じ規約($c_i > 0$ なら 1)。全体符号反転で cut は不変なので、どちらを 0/1 にするかは任意。
- **重み付き対応:**JIT 版(`modules/CIM.py:121-125`)では `edge_w` を足し込むことで重み付き MAX-CUT(K2000 等)にも対応。

4.1 ラウンドの流れ(まとめ図)



- **Step 1:**ポンプ $P_p = (k+1)dP \rightarrow g_0 = 2\kappa\sqrt{P_p}L$
- **Step 2-3:** $\mathbf{F}(n) = \sqrt{\eta}c + Jc$, $|\mathbf{F}|^2 = I_{in}$ (画像 1 本目)
- **Step 4:** $\sqrt{G_I} = \exp[\frac{1}{2}g_0(1 - \gamma I_{in})]$ (画像 2 本目の指数部)
- **Step 5-6:** $\mathbf{E}(n) = \sqrt{G_I} \mathbf{F}(n) + N_I$ (画像 2 本目 + ノイズ)
- **Step 7:**符号 $\rightarrow$ cut

5. 論文式と実装の対応(早見表)

論文式	内容	コード(modules/CIM.py)
Eq. (3a)	$c_i(k+1) = \sqrt{G_I}(\sqrt{\eta}c_i + \sum_j J_{ij}c_j) + N_I$	<code>c = sqrt_G_I*coupled_in + N_I (L341)</code>
Eq. (3b)	s_i の式(quadrature)	省略(結果に無関係, L291-292)
Eq. (14)	$g_0 = 2\kappa\sqrt{P_p}L, g = g_0(1 - \gamma I_{in}), G_I = e^g$	<code>g0=2κ√(P_p)L (L319), half_g=%g0(1-γI_in) (L332)</code>
Eq. (15)	$I_{in} \approx (\sqrt{\eta}c_i + \sum_j J_{ij}c_j)^2$	<code>coupled_in=√η·c+Jc; I_in=coupled_in² (L326-327)</code>
Eq. (6)	$\sigma_I^2 = (2 - \eta)G_I/4 \cdot BW$	<code>noise_const=√((2-η)·0.25·BW·ħω) (L298), N_I=... ·√G_I (L337)</code>

6. パラメータ一覧(論文 Section 3 / main)

modules/CIM.py:577-593 の設定値:

パラメータ	記号	値	役割
非線形定数	κ	130 W ^{-1/2} m ⁻¹	PSA 利得係数 g_0 を決める
PSA 媒質長	L	0.05 m (5 cm)	同上
飽和係数	γ	42.09 W ⁻¹	強信号で利得を頭打ちにする(β)
ループ損失	—	11 dB	$\eta = 10^{-1.1} \approx 0.0794$ (自己項 $a = \sqrt{\eta}$)
システム帯域	BW	1 GHz	ノイズ分散 Eq.(6) に乗る
一光子エネルギー	$\hbar\omega$	1.28 × 10 ⁻¹⁹ J	ノイズ式の単位換算(1550 nm)
ポンプ増分	dP	0.05 mW/round	ランプ速度($P_p = (k+1)dP$)
結合係数	—	-0.03	G22 辺の J_{ij} (反強磁性)
総ラウンド数	—	1500	焼きなまし長

発振しきい値は $P_{th} = (\ln(1/\eta)/(2\kappa L))^2 = 38.0$ mW。 P_p は round 1 の 0.05 mW(しきい値のはるか下)から round 1500 の 75 mW(しきい値の約 2 倍)まで連続的にスweepされ、これが焼きなましに相当する。

7. 初期条件・読み出し・実装上の工夫

- **初期条件**: 全パルスは vacuum 状態 $c(0) = 0$ から始まる (modules/CIM.py:293)。最初のノイズ N_I が種となって自発的に立ち上がる。
 - **スピン読み出し**: amplitudes_to_solution (L198-207) で $c_i > 0 \Rightarrow 1$ 、 $c_i \leq 0 \Rightarrow 0$ 。全体反転で cut 不変。
 - **検算**: run_all_checks (modules/verify.py) で独立実装と突き合わせて cut を二重チェック (L640-641)。
 - **高速化(3つ)**: 1. **Numba JIT** + prange : trial 単位で CPU コアに並列分散 (simulate_cim_batch, L49-138)。各 trial は独立なので競合なく並列化できる。 2. **融合ループ**: coupled_in $\rightarrow$ I_in $\rightarrow$ $\sqrt{G}$ _I $\rightarrow$ noise $\rightarrow$ c を 1 ループに融合し中間配列のアロケーションを削減 (L113-119)。 3. **定数の事前計算**: sqrt_eta, noise_const, half_g0などを毎ラウンドではなくループ外/ラウンド頭で 1 回だけ計算。
 - **2つの実行パス** (simulate_cim, L210-374):
 - **Fast path** (wandb_log=False): JIT 版 simulate_cim_batch を直接呼ぶ (本番・チューニング用)。
 - **Slow path** (wandb_log=True): numpy で 1 ラウンドずつ回し、wandb に各種メトリクスを記録 (デバッグ・可視化用)。
 - **振幅軌跡版** (simulate_cim_with_trajectory, L520-571): 指定ラウンドで $c(k)$ と cut を記録し、振幅の立ち上がり可視化に使う。
-

8.1 行まとめ

CIM の 1 ラウンドは、「① 損失付きで前振幅を残し他パルスから結合入力を集めて入力場 $\mathbf{F}(n)$ を作り (画像 1 本目)、② それにポンプ利得 $\exp[\alpha p(n)(1 - \beta|\mathbf{F}|^2)]$ をかけ飽和でロックし (画像 2 本目)、③ ノイズを足して次振幅 $\mathbf{E}(n)$ を確定、④ 符号で cut を測る」という流れであり、コードの Step 1~7 がこの各段に一対一で対応している。